Apellidos: ..... Nombre: ..... Grup: .....

Examen EP2 (*Temas 8, 9, 10, 11, 12, 13 y 14)*

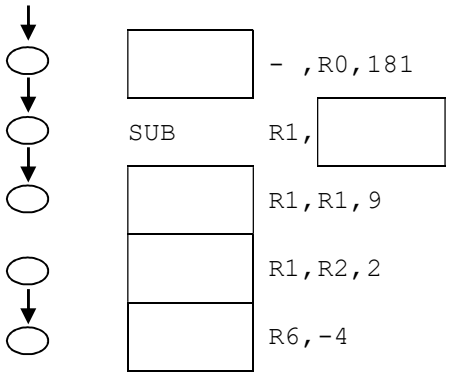
- Duración del examen: 2 horas y 30 minutos.
- La solución se tiene que escribir en el espacio reservado para ello en el propio enunciado.
- No podéis utilizar calculadora, móvil, apuntes, etc.
- La solución se publicará en Atenea mañana y las notas antes del XX de XXXX del 2025

Ejercicio 1 *(1 puntos)*

Dado el siguiente fragmento de código en C (el código no tiene que hacer algo útil), indicad como se implementaría en un procesador que use la UPG del procesador **Harvard uniciclo**. Primero utilizando una Unidad de Control de **propósito específico** (UCe) y luego utilizando una unidad de control de **propósito general** (UCG). Todos los datos son **naturales**.

```
if (R0<=181) {
      R1=R0-R1+9;
} else {
      R1=R2-2;
}
R6=-4;
```

a) Completad el fragmento del grafo de estados de la UC de **propósito específico** para que junto con la UPG del procesador **Harvard uniciclo** formen un procesador que realice la funcionalidad descrita en los fragmentos de código anteriores. Indicad los arcos que faltan, las etiquetas de los arcos (z, !z, o nada) y completad las casillas de cada palabra de control que se especifica con mnemotécnicos a la derecha de cada nodo del grafo. *(0.5 puntos)*



b) Completad el fragmento de programa en lenguaje ensamblador SISA para realice la funcionalidad descrita en el fragmento de código en C (el código no tiene que hacer algo útil). El código SISA ya escrito siempre utiliza el registro `R7` para valores temporales. En las comparaciones, hay que interpretar los datos como valores **naturales**. Rellenad la parte subrayada que falta. No se pueden usar etiquetas. *(0.5 puntos)*

@Mem	
0x0000	_____ R7, 0xB5
0x0002	MOV___ R7, ____
0x0004	CMP___ R7, R0, R7
0x0006	B____ R7, ____
0x0008	SUB R1, _____
0x000A	_____ R1, R1, 9
0x000C	B____ R7, ____
0x000E	_____ R1, R2, ____
0x0010	_____ R6, -4

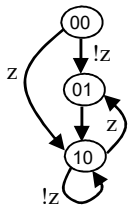
Ejercicio 2 *(0.9 puntos)*

Completa la siguiente tabla ensamblando las instrucciones en ensamblador SISA o desensamblando las instrucciones en lenguaje máquina según sea necesario. Indica poniendo **NA** en la casilla aquellos casos en los que la instrucción no corresponda al lenguaje SISA.

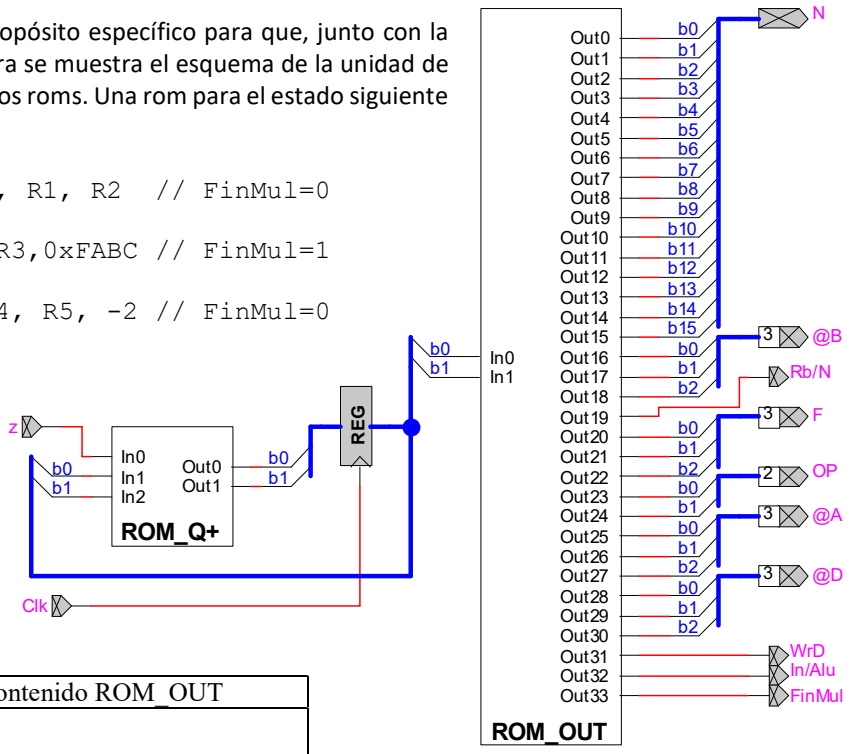
Lenguaje máquina SISA	Lenguaje ensamblador SISA
`0x022E`	
	`BZ R5, -3`
`0x3877`	

Ejercicio 3 *(0.8 puntos)*

Dado el siguiente grafo de estados de una UC de propósito específico para que, junto con la UPG básica, formen un PPE que lo ejecute. En la figura se muestra el esquema de la unidad de control correspondiente a una implementación con dos roms. Una rom para el estado siguiente (ROM_Q+) y otra rom para las salidas (ROM_OUT).




```
AND R0, R1, R2  // FinMul=0
MOVEI R3,0xFABC // FinMul=1
XORI R4, R5, -2 // FinMul=0
```



Indica el contenido de las ROMS en hexadecimal. Utilizad el valor 0 para los bits que sean x. Si los contenidos no están en hexadecimal no se corregirá el ejercicio.

@ROM	Contenido ROM_Q+
@0	
@1	
@2	
@3	
@4	
@5	
@6	
@7	

@ROM	Contenido ROM_OUT
@0	
@1	
@2	
@3	

Ejercicio 4 *(0.9 puntos)*

Indicad qué cambios se producen en el estado del computador SISC Von Neumann después de ejecutar cada una de las instrucciones de la tabla suponiendo que antes de ejecutarse cada una de ellas el PC vale `0x26AC`, el contenido de todos los registros pares es `0x6789` y el de los registros impares es `0x0123`, el byte contenido en todas las direcciones pares de la memoria es `0x64` y el de todas las impares es `0x32` y el contenido de todos los puertos de entrada es `1` y el de los de salida es `2`. Utilizad la notación MEM_b[0x...]=0x... y/o MEM_w[0x...]=0x... para indicar cualquier cambio en la memoria y PORTIN[0x...]=0x... para los puertos de entrada y PORTOUT[0x...]=0x... para los de salida.

Instrucción a ejecutar	Cambios en el estado del computador
`ST 9(R2), R5`	
`BZ R0, -6`	
`MOVHI R6, 94`	

Apellidos: ..... Nombre: ..... Grup: .....

Ejercicio 5 *(2.2 puntos)*

El programa en ensamblador SISA que se muestra a continuación se ha traducido a lenguaje máquina situando la sección de datos (.data) a partir de la dirección `0x0800` y la sección de código (.text) a partir de la dirección `0xA000`.

```
.data
       long=4
       A: .word 0x000F, 0xAB00, 0x0100,0xFFFE
       B: .byte 0X03, 0x90, 0x80, 0xFC
       C: .space 100
.text
       MOVI  R3, long
       MOVI  R1, lo(A)
       MOVHI R1, hi(A)
       MOVI  R2, lo(B)
       MOVHI R2, hi(B)
       MOVI  R7, 0
       MOVHI R7, 0x80
loop:  LD    R4, 0(R1)
       LDB   R5, 0(R2)
       SUB   R6, R4, R5
       AND   R0, R6, R7
       BNZ   R0, neg
pos:   ST    20(R1), R6
       BZ    R0, cont
neg:   ST    20(R1), R5
cont:  ADDI  R1, R1, 2
       ADDI  R2, R2, 1
       ADDI  R3, R3, -1
       BNZ   R3, loop
.end
```

Una vez ensamblado y cargado el programa en la memoria y antes de comenzar su ejecución:

a) ¿A qué direcciones de memoria corresponden las etiquetas o direcciones simbólicas siguientes? *(0.4 puntos)*

`C=0x` `loop=0x`

b) ¿Cuál es la dirección de memoria y su contenido donde han quedado almacenadas las siguientes instrucciones una vez cargado el programa? *(0.6 puntos)*

Instrucción	@ memoria y contenido
`LDB R5, 0(R2)`	Mem_w[0x] = 0x
`AND R0, R6, R7`	Mem_w[0x] = 0x
`BNZ R3, loop`	Mem_w[0x] = 0x

c) Una vez ejecutado el programa en el SISC von Neumann, completa la siguiente tabla con el contenido de la memoria en hexadecimal (no es necesario poner **0x** del valor). Las posiciones de memoria que no se pueda saber su contenido dejadlas en blanco. *(0.5 puntos)*

@memoria	Valor
0x0800	
0x0801	
0x0802	
0x0803	
0x0804	
0x0805	
0x0806	
0x0807	

@memoria	Valor
0x0808	
0x0809	
0x080A	
0x080B	
0x080C	
0x080D	
0x080E	
0x080F	

@memoria	Valor
0x0810	
0x0811	
0x0812	
0x0813	
0x0814	
0x0815	
0x0816	
0x0817	

@memoria	Valor
0x0818	
0x0819	
0x081A	
0x081B	
0x081C	
0x081D	
0x081E	
0x081F	

d) Si se ejecuta el código mostrado, ¿Cuántas instrucciones se ejecutan, cuántas son lentas y cuántas son rápidas en la Harvard multiciclo? Si se ejecuta el código en las tres versiones de los computadores SISC (Harvard uniciclo, el Harvard multiciclo y el Von Neumann) ¿Cuántos ciclos tarda en ejecutarse el código en cada una se los tres computadores? ¿Cuánto tarda en ejecutarse el código en el Harvard uniciclo, en el Harvard multiciclo y en el Von Neumann suponiendo que los tiempos de ciclo son 3.000, 2000 y 1.000 u.t. respectivamente? *(0.7 puntos)*

Nº de instruc. ejecutadas =

Nº de ciclos (H. uniciclo)= Nº de ciclos (H. multiciclo)= Nº de ciclos (Von Neumann)=

Tejec (Harvard uniciclo) = Tejec (Harvard multiciclo) = Tejec (Von Neumann) =

Ejercicio 6 *(1 punto)*

Cada uno de los apartados pregunta sobre un ciclo concreto de la ejecución de varias instrucciones en el SISC Von Neumann. Escribid el contenido del registro IR, el contenido de la ROM_OUT y el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**.** Para cada apartado/fila se indica el nodo/estado de la UC en ese ciclo y la instrucción (en ensamblador) que está almacenada en el IR en ese ciclo. Podéis ver el grafo de estados de Moore de la UC en el anexo. Suponed que el contenido de todos los registros, `Rk` para k=0,...7, antes de ejecutarse cada instrucción es 0.

a) Indica el contenido del registro IR. Utilizad el valor 0 para los bits que sean x (solo para este apartado). *(0.2 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Valor del IR** (en hexadecimal)
D	`JALR R1,R2`	0x
Stb	`STB 3(R4),R5`	0x

b) Indica el contenido de la ROM_OUT en hexadecimal usando las conexiones en el orden que están en el anexo. Utilizad el valor 0 para los bits que sean x (solo para este apartado). *(0.4 puntos)*

Nodo / Estado (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Contenido ROM_OUT** (en hexadecimal)
D	`JALR R1,R2`	0x
Stb	`STB 3(R4),R5`	0x

c) Indica el valor de los bits de la **palabra de control** que genera el bloque **SISC CONTROL UNIT** durante el ciclo a que hace referencia cada apartado**. Poned x siempre que no se pueda saber el valor de un bit** (ya que no podemos suponer cómo se han implementado las x en la ROM_OUT). *(0.4 puntos)*

Apartado	**Nodo / Estado** (Mnemo Salida)	**Instrucción en IR** (en ensamblador)	**Palabra de Control**																		
			@A	@B	Pc/Rx	Ry/N	OP	F	P/I/L/A	@D	WrD	Wr-Out	Rd-In	Wr-Mem	Ldlr	LdPc	Byte	Alu/R@	R@/Pc	N (hexa)	ADDR-IO (hexa)
a	D	`JALR R1,R2`																			
b	Stb	`STB 3(R4),R5`																			

Ejercicio 7 *(1.2 puntos)*

Especificad el **camino crítico** (indicando la suma ordenada de los tiempos de propagación de los bloques por los que pasa) y calculad el **tiempo de ciclo mínimo** para que el computador **SISC Von Neumann** pueda ejecutar correctamente el tipo de instrucción SISA que se indica en cada apartado (este sería el tiempo de ciclo mínimo del computador si solo ejecutara instrucciones como la indicada u otras que requieran menor tiempo). No tenéis que añadir ningún porcentaje de seguridad en el cálculo del tiempo de ciclo mínimo. Suponed que los tiempos de propagación de los bloques que forman el computador son los siguientes:

T_p(ROM_Q+) = 60 u.t.

T_p(ROM_OUT) = 80 u.t.

T_p(MUX-2-1) = 50 u.t.

T_p(MUX-4-1) = 100 u.t.

T_p(REG) = 100 u.t. // Tiempo de propagación de un registro.

T_p(REGFILE) = 200 u.t. // Tiempo de lectura del banco de registros

T_p(ALU-slow) = 800 u.t. // Tp de la ALU para las operaciones/funciones lentas: ADD, SUB, CMP*.

T_p(ALU-quick) = 400 u.t. // Tp de la ALU para las operaciones/funciones rápidas: cualquier otra distinta de ADD, SUB, CMP*.

T_{acc}(MEMORY) = 1000 u.t. // Tiempo de acceso (para la lectura o escritura) a la memoria

T_p(AND-2) = T_p(OR-2) = 20 u.t.

T_p(NOT) = 10 u.t.

El tiempo de propagación de un bloque combinacional (T_p) y el tiempo de acceso a memoria para realizar una lectura (T_{acc}) es el tiempo desde que están estables todas las entradas necesarias hasta que se estabilizan las salidas requeridas al valor correcto para las entradas aplicadas. Desconocemos como se han implementado internamente los bloques (y podría ser de forma diferente a los vistos en clase). Recordad que un registro con señal de carga (Ld), REGwLd, está construido con un REG y un MUX-2-1 (no os damos el esquema interno del REGwLd, porque lo tenéis que saber).

Apellidos: ..... Nombre: ..... Grup: .....

a) T_c correspondiente al nodo de **D** (decode).

T_c(D)=

b) T_c correspondiente al nodo de **Addi**.

T_c(Addi)=

c) T_c correspondiente al nodo de **Ldb**.

T_c(Ldb)=

Ejercicio 8 *(2 puntos)*

Apartado a)

Vamos a analizar diferentes aspectos de la ejecución, en el computador SISC Von Neumann, del siguiente fragmento de código:

```
MOVI R7, -2
SHA  R0, R1, R7
```

a.1) Si antes de comenzar a ejecutar la primera instrucción el estado del computador es el siguiente:

`PC=0x348E,  R0=0x348E,  R1=0x82A5,  R7=0x0002`

¿Completad el estado del computador que queda después de ejecutada la segunda instrucción? *(0.4 puntos)*

`PC=0x  R0=0x  R1=0x  R7=0x`

MEM_b[0x348E]=0x MEM_b[0x348F]=0x

a.2) Sea Nc el número de ciclos que tarda en ejecutarse el código. ¿Cuánto vale Nc? *(0.1 puntos)*

Nc=

a.3) Completad las 4 filas de la siguiente tabla que hacen referencia a los ciclos de ejecución 3, 4, 5 y 6 (los ciclos de ejecución se numeran desde el 1 al Nc). Podéis ver los mnemotécnicos de salida de cada Nodo en el grafo de la chuleta. En la columna Acciones usad el lenguaje de transferencia de registros de la documentación de la asignatura (o una notación igual de precisa). Recordad que en la Palabra de control compactada solo hay que expresar el valor de los campos que son significativos en ese ciclo (no hay que indicar los campos que valen x ni las señales de permiso de modificación del estado que valen 0). Completad esta tabla para el **caso general** del código **MOVI Rd,N8** y **SHA Rd,Ra,Rb**. *(0.4 puntos)*

Ciclo	**Nodo** Mnemo Salida	**Acciones**	**Palabra de control compactada**
3			
4			
5			
6			

Apartado b)

Completad el diseño del SISC Von Neumann para que pueda ejecutar, además de las 25 instrucciones originales SISA, la nueva instrucción

```
SHAI Rd, Ra, N5
```

que al ejecutarse modifica el estado del computador de la misma forma que lo hace la instrucción SHA pero siendo el segundo operando un inmediato de 5 bits que codifica un número entero en complemento a 2. Con esta nueva instrucción podemos hacer lo mismo que en el SISA original requiere dos instrucciones (como las dos del fragmento de código del *Apartado a*).

La nueva instrucción se tiene que poder ejecutar sin efectuar ningún cambio en el computador excepto el contenido de la ROM_Q+ y el de la ROM_OUT. La nueva instrucción se tiene que ejecutar en el mínimo número de ciclos. La ejecución de las 25 instrucciones SISA originales se hace exactamente igual que antes de introducir la nueva instrucción. Se pide:

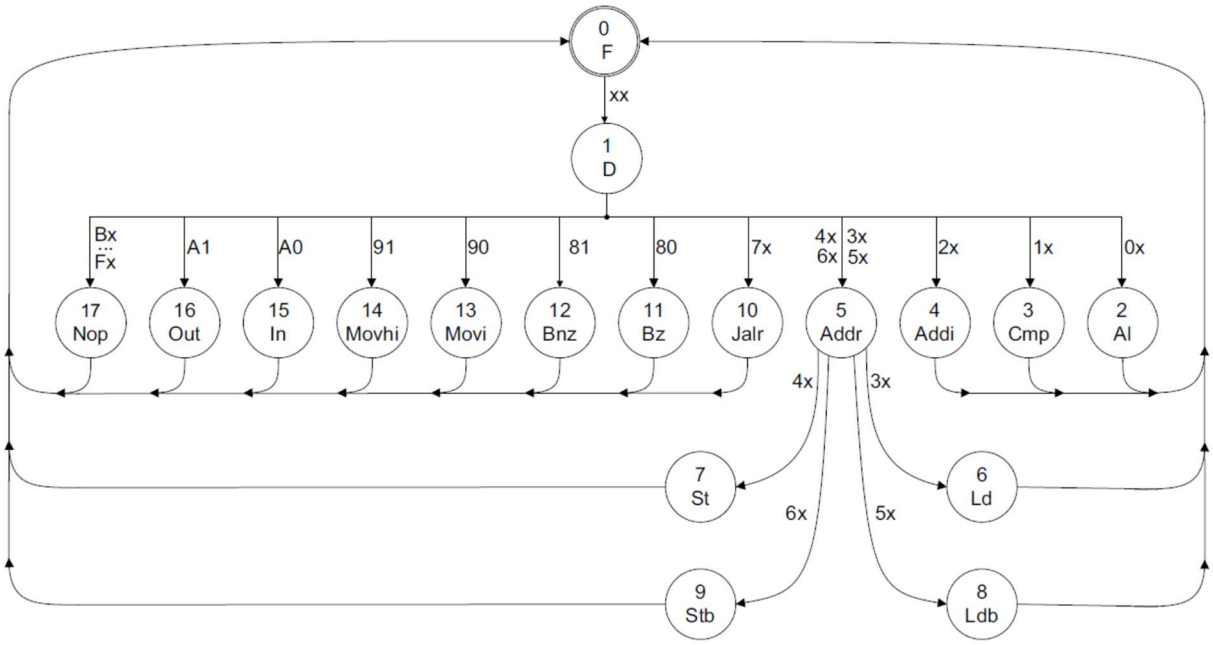
b.1) Proponed el formato y la codificación en binario de la nueva instrucción (cada bit puede valer 0, 1, a, b, d, n, x) y escribid la semántica usando un fragmento de código en un lenguaje de alto nivel tipo C, como el empleado para esto en la documentación. *(0.2 puntos)*

Codificación:

Sintaxis: SHAI Rd, Ra, N5

Semántica:

b.2) Modificad el grafo de estados de la unidad de control (añadiendo nodos y arcos y/o modificando los ya existentes) dibujando sobre y/o al lado del siguiente grafo original. *(0.2 puntos)*



b.3) Completad la tabla usando una fila para cada nuevo nodo que hayáis añadido al grafo (no son más de 2 nodos). *(0.2 puntos)*

Nodo			
Estado (decimal)	**Mnemo salida**	**Acciones**	**Palabra de control compactada**

b.4) Indicad la dirección o las direcciones (en binario, con x cuando sea posible para referirnos a más de una dirección) de la ROM_Q+ y su contenido (en hexadecimal) para implementar correctamente el arco que va del nodo/estado 1 (con mnemotécnico de salida D) al primer nuevo nodo usado para implementar la nueva instrucción. *(0.2 puntos)*

	Dirección o direcciones (en binario)	**Contenido** (en hexa)
Arco del nodo 1 (D) al nodo		

b.5) Completad (poniendo 0, 1 o x en cada bit) la siguiente tabla que especifica, mediante una fila por cada nuevo nodo añadido al grafo de estados de la unidad de control (no son más de 2 nodos), la dirección (en decimal) de la ROM_OUT y su contenido para que se ejecute correctamente la nueva instrucción, poniendo el máximo número de x posibles. En la columna de la derecha poned el mnemotécnico de salida que habéis usado para nombrar cada nuevo nodo en el grafo del apartado b.2). *(0.3 puntos)*

ROM	Bnz	Bz	WrMem	RdIn	WrOut	WrD	LdIr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	P/I/L/A_1	P/I/L/A_0	OP_1	OP_0	MxN_1	MxN_0	MxF	F_2	F_1	F_0	$Mx@D_1$	$Mx@D_0$	**Nodo (Mnemo Salida)**